

DeFa on Arc — Ignyte Stablecoins Commerce Stack Challenge Submission

Submission tracks

- **Track 2 — SME Trade Finance & Working Capital** (*primary*)
- **Track 4 — Agentic Economy** (*secondary — the KAM/CAD/CRO approval chain is agent-ready*)

Repository: <https://github.com/Mate-Sol/The-Stablecoins-Commerce-Stack-Challenge-Arc-Circle-> **Live demo:** <https://defa-arc-hackathon.invoicemate.net> (access code `654321`) **Chain:** Arc Testnet (chain id `5042002`)

Team Background

DeFa is built by the InvoiceMate team. Two-person build for this submission:

- **Ibrahim Ansari — CEO.** Trade finance + fintech background. Owns product framing, regulatory framing, and the credit-workflow spec (KAM → CAD → CRO → Legal).
- **Hamza Anjum — CTO.** Full-stack + Solidity. Shipped the entire submission — `payfi_v1` contract set, Node/ethers backend, React lender UI, and the admin/PSP portal.

Backing engineering pod at InvoiceMate provides code review and QA support outside the sprint.

Problem Statement

Cross-border SME trade finance is roughly **\$2 trillion undersupplied globally** (ADB Trade Finance Gaps Report, 2024). In the corridors we target — UAE ↔ Pakistan, UAE ↔ South Africa, Gulf ↔ East Africa — three specific frictions strand working capital:

1. **Settlement latency.** A PSP paying a supplier in Karachi against an invoice financed in Dubai waits 2–5 business days for SWIFT to clear. The SME is out of stock; the PSP carries the settlement risk on its balance sheet the whole time, and that cost gets priced back into every facility.
2. **FX drag on the drawdown leg.** When a facility is priced in USD, drawn in AED, and settled in PKR, the PSP eats the spread on every drawdown. Multiplied over hundreds of drawdowns per month, 200–400 bps of margin evaporates before it reaches the LP.
3. **No shared credit primitive.** Every bank runs its own KAM → CAD → CRO workflow in a separate silo. A PSP that's been underwritten by one bank has to redo the entire KYB and credit review to onboard with the next. There is no reusable on-chain primitive that says *"this PSP is underwritten, this pool is funded, here's the automated waterfall."*

Why USDC on Arc solves it

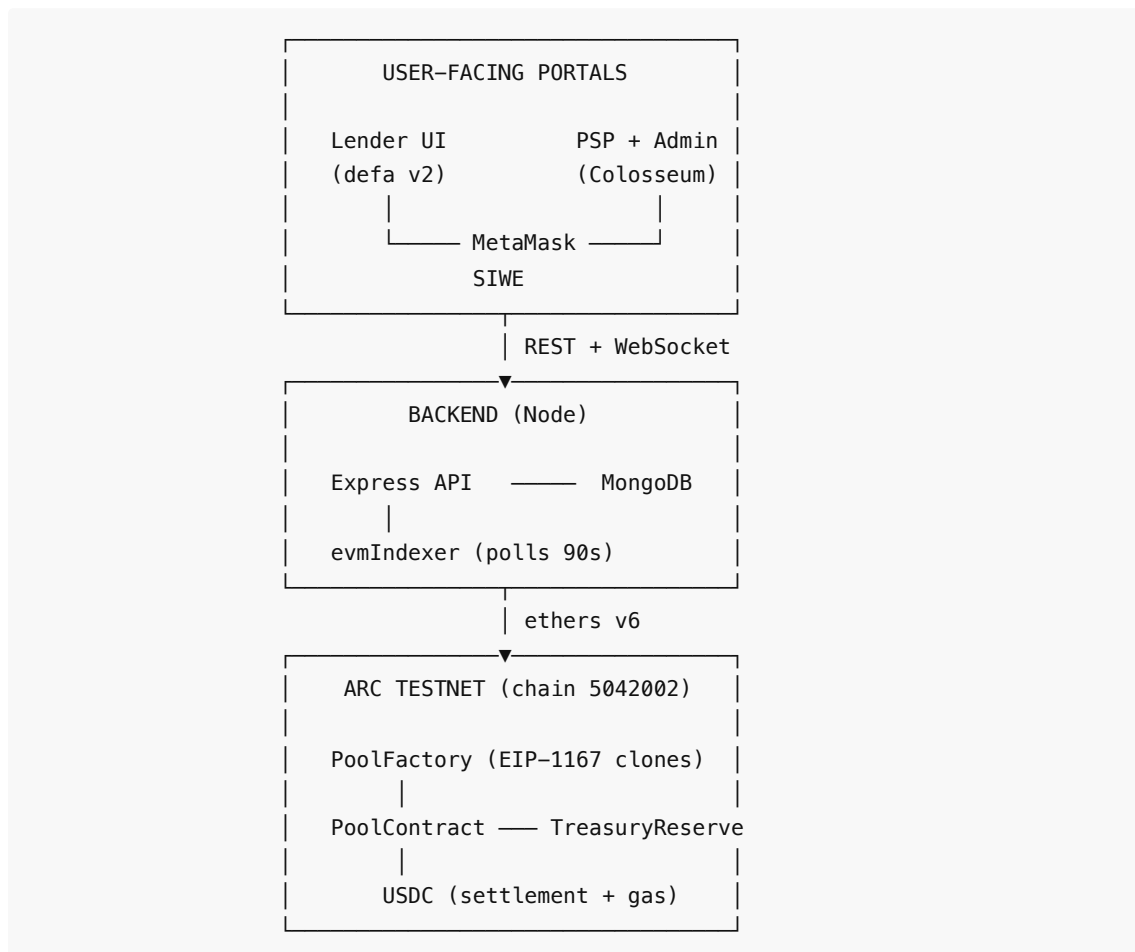
The three frictions collapse to one physics problem: **money should move at the speed of software, and underwriting should be a contract, not a PDF.** USDC gives us programmable settlement — but USDC deployed on a general-purpose L1 comes with two frictions of its own:

- The user needs a separate gas asset. An SME with USDC has no ETH to sign a repayment.
- 6-decimal stablecoin math gets bolted onto 18-decimal-native WAD primitives.

Arc removes both. USDC is the gas asset. 6-decimal math is first-class. Compliance hooks are native. For cross-border trade finance, this is the difference between a demo and a rail a PSP can actually run production volume through.

Technical Architecture

System overview



Full Mermaid diagrams (top-level graph, PSP → KAM → CAD → CRO sequence, contract class) live in [docs/ARCHITECTURE.md](#).

Contract layer — payfi_v1

- **PoolFactory** — EIP-1167 minimal-proxy factory. `approvePsp(address)` gates who can operate a pool. `createPool(params)` clones a fresh `PoolContract` initialized with risk-envelope bounds (APR ceiling, utilisation cap, penalty rate, tenor limit).
- **PoolContract (per-facility)** — deposit / withdraw for LPs, `executeDrawdown(ref, receiver, amount, days)` for PSPs, `repay(ref)` waterfall (principal → utilization fee → penalty → protocol split → LP yield), `claimYield` + `claimPrincipal` for LPs, `declareDefault` fallback that draws from `TreasuryReserve`.
- **TreasuryReserve** — protocol fee sink + insurance reserve.
- **MockStablecoin** — 6-decimal USDC surrogate for testnet. Swaps 1:1 to real USDC on mainnet.

Uses OpenZeppelin v5.x (`AccessControl` , `ReentrancyGuard` , `Clones` , `SafeERC20`).
`_disableInitializers` on the implementation contract. WAD fixed-point math with invariant tests. ~17K LOC of Foundry test coverage across 26 test files (unit, invariant, adversarial, differential, fuzz, gas-profile).

Backend layer — Node / Express / ethers v6

- 18 route files: auth (SIWE + JWT), pool discovery, facility lifecycle, PSP KYB, KAM/CAD/CRO/Legal/on-chain-admin approvals, faucet, admin build-tx endpoints.
- `poolServiceEvm` — reads live pool state and builds calldata for user wallets to sign.
- `walletAuthEvm` — SIWE (EIP-4361) with nonce store and domain binding.
- `evmIndexer` worker — polls `PoolCreated` , `Deposit` , `DrawdownExecuted` , `Repaid` on a 90s window and mirrors state into Mongo.
- `_withRetry` on all RPC calls — coalesces `CALL_EXCEPTION`, `ECONNRESET`, and missing-revert-data errors that Arc's testnet RPC surfaces under burst load.

Frontend layer — two portals

- **Lender UI** (`client/`) — React 19 + Vite + Tailwind v4 + wagmi v2 + viem v2 + RainbowKit v2.
Browse pools, deposit USDC (2-step approve + deposit), watch utilisation live, redeem principal + yield.
- **PSP + admin portal** (`client-legacy/`) — PSPs submit KYB and request facilities;
KAM/CAD/CRO/Legal walk each facility through approvals; on-chain admin signs the two-transaction pool bootstrap (`approvePsp` + `createPool`).

End-to-end lifecycle

- | | |
|-------------------------------------|---|
| 1. PSP signs up | → KYB submission created |
| 2. PSP requests facility | → <code>Facility.status</code> = <code>KAM_REVIEW</code> |
| 3. KAM approves | → → <code>CAD_REVIEW</code> |
| 4. CAD approves | → → <code>CRO_REVIEW</code> |
| 5. CRO approves + terms | → → <code>AWAITING_POOL_INIT</code> |
| 6. Onchain admin signs | → <code>factory.approvePsp</code> + <code>factory.createPool</code> |
| 7. <code>evmIndexer</code> picks up | → pool visible on <code>/pools</code> within 90s |
| 8. LP deposits | → approve + deposit |
| 9. PSP executes drawdown | → server signs <code>AGENT2</code> , USDC to receiver |
| 10. PSP repays | → principal + utilisation fee + penalty |
| 11. LP redeems | → <code>claimYield</code> + <code>claimPrincipal</code> |

Deployed on Arc Testnet (chain 5042002)

Contract	Address
PoolFactory	0x4e39880B43f9a83586a2aC75a01dff779Eb958c0
MockUSD (6-decimal USDC surrogate)	0x2b2037760695772770182C84dFeE2b9594526c7f
TreasuryReserve	0xcC3a9A71532a1402Ab57742C22661eE6e96102e5

Three demo facilities are live on-chain right now: Mercury Settlements USDC Facility (12% APR, medium risk), Aurum Cross-Border Corridor (6% APR, low risk), Meridian FX Working Capital (14% APR, medium risk).

Test coverage — 38 integration tests, all green on real Arc Testnet

Suite	Result
e2eFlows.test.js — auth, marketplace, deposit build-tx, faucet, gates	18/18
lifecycleFlows.test.js — PSP → KAM → CAD → CRO → onchain admin gating	13/13
onchainFlows.test.js — real approvePsp + createPool + faucet mint + BE state read	7/7

Track 4 mapping (Agentic Economy)

The KAM → CAD → CRO → Legal chain is workflow, not judgment — each seat's decision is bounded by structured inputs (KYB data, credit-memo terms, envelope constraints). That means each approval seat can be run by a deterministic agent:

- **KAM agent** — verifies KYB completeness + PSP flow history.
- **CAD agent** — recomputes credit exposure against portfolio limits.
- **CRO agent** — validates term envelope against the risk framework.
- **Legal agent** — checks facility-agreement schema.

The on-chain-admin remains a human multisig signer (deliberate custody choice, not an agent gap). The backend already routes everything through structured `build-tx` endpoints — swapping in agents is a config change, not an architecture change.

Circle Products Integrated

- **Public Faucet** (<https://faucet.circle.com/>) — funded all testnet wallets (deployer, agent, demo LP/PSP) with canonical Circle-issued testnet USDC on both Arc Testnet and Polygon Amoy.
- **Canonical Arc Testnet USDC** — the pool settlement asset is now the Circle-issued canonical USDC contract on Arc, not our earlier `MockStablecoin` clone. This is the same asset users will settle in on mainnet — no post-audit swap-out needed.
- **Circle Console** — used for wallet funding, API-key management, and to reference canonical contract addresses.

On our roadmap (Circle products we plan to integrate next)

- **CCTP** — Phase 2 (Q4 2026). Cross-chain drawdowns: LP funds pool on Arc, PSP draws USDC on Ethereum L1 supplier wallet, waterfall repay routes back to Arc. Our backend already routes through structured `build-tx` endpoints, so adding a CCTP-message step is a targeted addition, not a rewrite.
- **Gas Station** — Phase 2 (Q4 2026). Sponsored gas for SME PSPs so they never need to hold USDC just to sign a drawdown. Hook already exists in our AGENT2 executor path.
- **Programmable Wallets** — Phase 3 (H1 2027). Optional embedded-wallet onboarding for SMEs that don't want to manage MetaMask themselves, alongside our current SIWE + external-wallet flow.

Launch Roadmap

Phase 1 — Sprint (through Aug 2026)

- **Jul 13** — Ignyte submission (this).
- **Jul–Aug** — Pilot facility with a PSP partner in the UAE ↔ Pakistan corridor.
- **Aug** — Engage third-party audit for the `payfi_v1` contract set.

Phase 2 — Q4 2026

- Migrate `MockStablecoin` → native USDC on Arc mainnet.
- `PoolFactory v2` — configurable per-pool risk envelopes, protocol-fee auction, multi-asset support.
- **Circle CCTP integration** for cross-chain drawdowns (LP funds on Arc, PSP draws to Ethereum L1 supplier wallet, waterfall repay flows back to Arc).

Phase 3 — H1 2027

- Three live facilities across UAE ↔ Pakistan, UAE ↔ South Africa, and Gulf ↔ Kenya corridors.
- Target: **\$10M cumulative drawdown volume** across pilot facilities.
- Agent-driven KAM / CAD reviews live in production behind a human-approval fallback.

Phase 4 — H2 2027

- Open `payfi_v1` to third-party PSPs — any PSP can spin up a pool without running the underwriting stack themselves.
- Multi-chain deployment (Arc + Polygon + Base) with CCTP-based liquidity balancing.
- Target: **\$50M cumulative drawdown volume**.

Circle Product Feedback

(Mandatory section per Arc / Circle challenge rules.)

What worked

- **USDC-as-gas is the right primitive.** We dropped the "get testnet ETH from a faucet before you can deposit" step from the lender UX entirely. One asset.
- **6-decimal-native math.** Our contract set uses 6-decimal WAD math throughout. On other EVM chains we've had to write per-asset decimal shims; on Arc it just drops in.
- **Testnet RPC uptime** was solid enough to run the full 38-test integration suite against real Arc Testnet many times per day without RPC being the flaky link.
- **Block explorer** on `testnet.arcsan.app` was fast enough that state changes were verifiable within a block of confirmation — critical for our debug loop.

Friction points

- **Faucet gating.** The "you need mainnet history to claim testnet USDC" gate is important for anti-sybil, but became a real hackathon blocker for fresh wallets. We ended up funding from a wallet with mainnet history after losing time to faucet hunting. A dedicated hackathon-code-based faucet would fix this cleanly.
- **MetaMask config docs.** Chain-id and native-currency configuration for MetaMask was thin — we traced values from block-explorer metadata. A "Add Arc Testnet to MetaMask" one-click button on `docs.arc.build` would save every team the same 20 minutes.
- **Canonical testnet USDC address.** We couldn't find an authoritative "canonical testnet USDC" address in the docs during the sprint window, so we shipped a `MockStablecoin` clone. Publishing a canonical testnet USDC address next to the RPC endpoints would remove this.
- **CCTP on testnet is not obvious to discover.** We wanted to demo cross-chain drawdown but couldn't locate the CCTP testnet contract map for Arc in the sprint window. If it exists, link it from the chain-info page; if not yet, a target date would help teams roadmap around it.

What we'd love next

1. **Native KYB attestation at the ERC-20 layer.** "This address is KYB'd" as a first-class contract attestation removes an entire allowlist-proxy pattern we currently deploy.

2. **A canonical "receivables / trade finance" reference implementation in `arc-examples`** . We'd contribute `payfi_v1` as a starting point.
3. **CCTP-native cross-chain drawdowns from a single pool**. LP funds on Arc, PSP draws to Ethereum L1 supplier wallet, waterfall repays flow back to Arc. Package this as a single Arc-side contract entry point and it's an enormous unlock for cross-border credit.
4. **A Circle-branded compliance oracle** that pool admins can reference during pool bootstrap ("only allow drawdowns to Circle-attested KYB'd receivers"). Materially easier for regulatory teams at partner banks to sign off on than an in-house allowlist.